# LessonForge — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

## Hint 1 — the pieces, and where to start

The same shape as ResearchPilot, for lessons:

- `src/schemas.py` — `SubTopics(sub_topics)`, `Section(sub_topic, explanation, source)`, `Lesson(topic, sections)`.
- `src/builder.py` — `build_sections(sub_topics, search)`: call `search(sub_topic)` (returns `(text, source)`) for each, build a `Section`. **Build this + the compiler first** — the trick is `search` is **passed in** (fake in tests, RAG in the pipeline).
- `src/lesson.py` — `compile_lesson` + `format_lesson` (numbered, de-duplicated citations).
- `src/planner.py` — `make_plan(model, topic)` with `with_structured_output(SubTopics)`.
- `src/rag/` — `build_library_store()` (chunk + embed, keep filename as `source` metadata) + `make_search(collection)` → `search(q) -> (text, source)`.

Build order: schemas → builder + lesson (test them) → planner → RAG → pipeline.

## Hint 2 — the citation logic, and the one lambda-free closure

`format_lesson` numbers sources in first-seen order; two sections from the same file share a number:

```
sources = []
for section in lesson.sections:
    if section.source not in sources:
        sources.append(section.source)
# each section: cite = sources.index(section.source) + 1
# then a "Sources:" list, each with its [number]
```

The pipeline is: `make_plan` → `build_library_store` → `make_search` → `build_sections` → `compile_lesson`. `make_search` returns a closure so `build_sections` never imports Chroma —

which is why you could test it with a fake `search`. Keep just the filename (`source.split("/")[-1]`) so citations read `rag_basics.md`. The whole file set is in the complete solution.